



Topic 1: MATLAB Basics

Jaber Taheri-Shakib

Research associate

Civil and Environmental Engineering

University of Waterloo | McMaster University

Function Basics

`out = fun(arg1, arg2,...)`

- Functions go behind the scenes to perform a set of operations/calculations and **return** the results (i.e. make an assignment to the output variable (`out`))
- To call a function, give its name (`fun`) followed by the **argument(s)** (`arg1, arg2,...`) that are **passed** to it in parentheses `()` [`x = sqrt(25)` → the output is 5].



Functions give us a “machine” to transform a set of input variables into a new output variable.

- Some of our arguments to the function are these input variables.
- Other arguments to the function are settings, that modify how the transformation happens.

MatLab's Built-In Functions

- MatLab has a rich collection of built-in functions to help formulate and solve mathematical and engineering problems.
- There are too many to list:
 - Trigonometric functions
 - Logarithmic/exponential functions
 - Complex functions
 - Rounding and remainder functions
 - Coordinate transformation functions
 - I/O Functions
 - String Functions
 - Specialized functions (signal processing, controls, etc.)
 - And many more...
- We'll be making use of many of them, but for now we'll just start with a few of the most functions we'll commonly use in this course.

Overwriting Functions

- Just like constants, functions can be overwritten by MatLab variables.
- MatLab will not stop you or notify you if you overwrite a function.
- The function can be made available again by using the command `clear` to wipe the matching variable out of memory.

⚠: MatLab always looks for variables first!
Be careful to choose variable names that do not match existing functions or constants that may be used in your code.



Overwriting Functions

<code>x = 25;</code> <code>out = sqrt(x)</code>	<code>out =</code> 5
--	-----------------------------

1

<code>sum = 10;</code> <code>numbers = [1 2 3];</code> <code>sum(numbers)</code>
--

<code>clear sum</code> <code>sum(numbers)</code>	<code>ans =</code> 6
---	-----------------------------

3

<code>numbers = [4 10 2 8];</code> <code>largestValue = max(numbers)</code> <code>averageValue = mean(numbers)</code> <code>totalValue = sum(numbers)</code>

<code>largestValue =</code> 10 <code>averageValue =</code> 6 <code>totalValue =</code> 24
--

2

Overwriting Functions

Question

A right triangle has two sides:

```
</> MATLAB
```

```
a = 3;
```

```
b = 4;
```

Use MATLAB to calculate the length of the third side using this formula:

$$c = \sqrt{a^2 + b^2}$$

Write the MATLAB code and find the value of `c`.

Answer

```
</> MATLAB
```

```
a = 3;
```

```
b = 4;
```

```
c = sqrt(a^2 + b^2)
```

Output

```
</> MATLAB
```

```
c =
```

```
5
```

Overwriting Functions

Question

A student wants to calculate the average of these numbers:

```
<> MATLAB  
  
data = [10 20 30 40];
```

But the student writes this code:

```
<> MATLAB  
  
mean = 100;  
  
data = [10 20 30 40];  
  
averageValue = mean(data)
```

What is the problem?

How can we fix the code?

Correct Code

```
<> MATLAB  
  
clear mean  
  
data = [10 20 30 40];  
  
averageValue = mean(data)
```

Output

```
<> MATLAB  
  
averageValue =  
  
25
```


User Communication Functions

```
1 year = input("What year is it?\n");
2 gradYear = year + 4;
3 fprintf("\nHello Class of %d!\n", gradYear);
```

Function Name: input

Arguments: string of text to prompt the user

Operation Performed:

1. Display provided text
2. Wait for the user to type something and hit return
3. Retrieve what was typed.

Output: the value typed by the user.

Function Name: fprintf

Arguments:

1. String of text to display, with formatted variable bins
2. Variables to include in display

Operation Performed:

1. Insert variables into format slots
2. Display provided text, as modified by step (1)

Output: number of bytes written (not collected)

`input ()` : Collecting Formatted Data from the User

We can gather information from our user and store that information in variables using the `input()` command.

```
1 year = input("What year is it?\n");
```

The default operation of this function allows us to gather ***scalar, numeric expressions***. To gather character variables, we need an additional arguments.

Argument Options	Operation Performed
<code>numVal = input("prompt")</code>	Assumes that the characters entered by the user form a numerical expression. Evaluates the expression and stores the result as a variable of type double.
<code>charVal = input("prompt", "s")</code>	Retrieves exactly what was entered by the user and stores the entry as a character array.

`fprintf()` : Formatted Printing for All Variables

How variables are reported in our output text is dependent on the class of variable. Let's take a look at our example from the first lecture:

```
1 year = input("What year is it?\n");
2 gradYear = year + 4;
3 fprintf("\nHello Class of %d!\n", gradYear);
```

: Formatting characters like `\n` (newline) and `\t` (tab) can also be used with the `fprintf` statement to shorten the print statement and clarify the output to the user.

Variable Type	Associated Classes	Format Code
Real Numbers	single, double	<code>%.•f</code> for decimal numbers <code>%.•e</code> for scientific notation
Integers	int8, int16, int32, int64	<code>%.•d</code> or <code>%.•l</code>
Characters	char	<code>%.•c</code> for single character <code>%.•s</code> for set of characters
Strings	string	<code>%.•s</code>
True/False	logical	must convert to numeric or string

: The value `•` specifies the precision of the reported numeric value. The value `*` specifies the minimum number of slots allotted for the printout (unused slots are set to the left and assigned a `' '` character).

```

1  sampInt = 42;
2  sampNum1 = 2.163;
3  sampNum2 = 646523421.123; sampChar =
4  'Samuel';
5  sampStr = "Sarah";
6
7
8  fprintf("The output is %d.\n", sampInt);
9  fprintf("The output is %f.\n", sampInt);
10 fprintf("The output is %14i.\n", sampNum1);
11 fprintf("The output is %.2f.\n", sampNum1);
12 fprintf("The output is %f.\n", sampNum2);
13 fprintf("The output is %.2e.\n", sampNum2);
14 fprintf("The output is %s.\n", sampChar);
15 fprintf("The output is %6.2s.\n", sampChar);
16 fprintf("The output is %.3s.\n", sampStr);
17 fprintf("The output is %f.\n", sampStr);

```

⚠: Print formatting **does** perform rudimentary type casting within numeric and string/character types, respectively. Print formatting **does not** perform casting between string and numbers. Remember to match your format specifier with the right variable!

The output is 42. ✓
 The output is 42.000000. ✓
 The output is 2.163000e+00. ✓
 The output is 2.163. ✗ 2.16
 The output is 646523421.123000. ✓
 The output is 6.47e+08. ✓
 The output is Sam. ✗ Samuel
 The output is Sa. ✓
 The output is Sar. ✓
 ✗ Error: %f cannot be used for the string "Sarah".

Formatted Output Examples

Question

Write a MATLAB script that asks the user to enter:

1. Their first name
2. The current year
3. The number of years remaining until graduation
4. Their current average mark

Then calculate the expected graduation year and display a formatted message.

Your output should look similar to this:

Use:

</> MATLAB

```
Hello Sarah!  
Current year: 2026  
Expected graduation year: 2030  
Current average mark: 87.50%
```

</> MATLAB

```
input()  
input(...,"s")  
fprintf()  
%d  
%.2f  
%s  
\n
```

```
name = input("Enter your first name: ", "s");  
  
currentYear = input("Enter the current year: ");  
  
yearsRemaining = input("Enter the number of years remaining until graduation: ");  
  
averageMark = input("Enter your current average mark: ");  
  
gradYear = currentYear + yearsRemaining;  
  
fprintf("\nHello %s!\n", name);  
fprintf("Current year: %d\n", currentYear);  
fprintf("Expected graduation year: %d\n", gradYear);  
fprintf("Current average mark: %.2f%%\n", averageMark);
```

Common Mathematical Functions

Function	Description	Example	
		Script	Command Window
<code>sqrt(x)</code>	Finds the square root of x	<code>sqrt(4)</code> <code>sqrt(1.75)</code>	2 1.32
<code>nthroot(x,n)</code>	Computes the n^{th} real root of x	<code>nthroot(-27,3)</code> <code>nthroot(64,6)</code>	-3 2
<code>abs(x)</code>	Finds the absolute value of x	<code>abs(-3)</code> <code>abs(2)</code>	3 2
<code>sign(x)</code>	Returns -1 if x is less than zero, a value of 0 if x equals zero, and a value of 1 if x is greater than zero.	<code>sign(-5)</code> <code>sign(3)</code> <code>sign(0)</code>	-1 1 0
<code>factorial(n)</code>	Computes $n!$, where n is a nonnegative integer	<code>factorial(4)</code> <code>factorial(0)</code>	24 1
<code>rem(x,y)</code>	Computes the remainder of x/y	<code>rem(25,4)</code> <code>rem(4,2)</code> <code>rem(9,5)</code>	1 0 4

Exponential Functions

Function	Description	Example	
		Script	Command Window
<code>exp (x)</code>	Computes the exponential e^x	<code>exp (1)</code> <code>exp (0)</code> <code>exp (log (5))</code>	e 1 5
<code>log (x)</code>	Computes the natural logarithm of x	<code>log (1)</code> <code>log (2)</code> <code>log (10)</code> <code>log (0)</code> <code>log (-5)</code>	0 0.693 2.3 -inf $1.61 + 3.14i$
<code>log10 (x)</code>	Computes the base 10 logarithm of x	<code>log10 (1)</code> <code>log10 (2)</code> <code>log10 (10)</code>	0 0.301 1
<code>log2 (x)</code>	Computes the base 2 logarithm of x	<code>log2 (1)</code> <code>log2 (2)</code> <code>log2 (10)</code>	0 1 3.32

⚠: MatLab includes negative and complex numbers in the domain of logarithm functions.

⚠: MatLab does NOT have a pre-defined `ln` function.

Trigonometric Functions

⚠: MatLab expresses angles and functions dealing with angles in radians by default.

Function	Description	Example	
		Script	Command Window
sin (x)	Finds the sine of the angle x , where x is given in <u>radians</u> .	<code>sin(pi/2)</code> ($\pi/2$) <code>sin(pi/6)</code>	0 0.5
cos (x)	Finds the cosine of the angle x , where x is given in <u>radians</u> .	<code>cos(pi/2)</code> <code>cos(pi/6)</code>	0 $\sqrt{3}/2$
tan (x)	Finds the tangent of the angle x , where x is given in <u>radians</u> .	<code>tan(pi/2)</code> <code>tan(pi/6)</code>	
sec (x)	Finds the secant of the angle x , where x is given in <u>radians</u> .	<code>sec(pi/2)</code> <code>sec(pi/6)</code>	
csc (x)	Finds the cosecant of the angle x , where x is given in <u>radians</u> .	<code>csc(pi/2)</code> <code>csc(pi/6)</code>	
cot (x)	Finds the cotangent of the angle x , where x is given in <u>radians</u> .	<code>cot(pi/2)</code> <code>cot(pi/6)</code>	

📖: To work with degrees instead of radians, append the name of the trigonometric function with a **d** (e.g. `sind(x)`).

⚠: Truncation issues can yield very large or small numbers when `Inf`, `-Inf`, or `0` would be more appropriate.

📖: Inverse trigonometric functions can be called by prepending the function with an **a** (e.g. `asin(x)` or `asind(x)`).

Rounding Functions

Function	Description	Note
<code>round(x)</code>	Rounds x to the nearest integer	n $n+1$
<code>fix(x)</code>	Truncates x to the nearest integer toward zero.	0
<code>floor(x)</code>	Rounds x to the nearest integer toward negative infinity.	0
<code>ceil(x)</code>	Rounds x to the nearest integer toward positive infinity.	0

Input	Function Calls			
	round	fix	floor	ceil
0.3	0	0	0	1
0.6	1	0	0	1
-0.3	0	0	-1	0
-0.6	-1	0	-1	0

$x = [0.3 \ 0.6 \ -0.3 \ -0.6];$

Question

Question

Use MATLAB to evaluate the following mathematical functions.

In the Command Window, define:

«» MATLAB

```
x = -7.8;  
y = 64;  
z = 25;  
theta = pi/6;  
values = [-0.6 -0.3 0.3 0.6];
```

Then answer the following using MATLAB functions:

1. Find the square root of `z`.
2. Find the 3rd root of `-27`.
3. Find the absolute value and sign of `x`.
4. Calculate `factorial(5)`.
5. Find the remainder of `25/4` and `9/5`.
6. Calculate `exp(2)`, `log(exp(2))`, `log10(1000)`, and `log2(16)`.
7. Calculate `sin(theta)`, `cos(theta)`, and `tan(theta)`.
8. For `values`, calculate `round`, `fix`, `floor`, and `ceil`.

Answer

Part	MATLAB Command	Expected Output
Square root of 25	<code>sqrt(25)</code>	5
3rd root of -27	<code>nthroot(-27,3)</code>	-3
Absolute value of -7.8	<code>abs(-7.8)</code>	7.8
Sign of -7.8	<code>sign(-7.8)</code>	-1
Factorial of 5	<code>factorial(5)</code>	120
Remainder of 25/4	<code>rem(25,4)</code>	1
Remainder of 9/5	<code>rem(9,5)</code>	4
Exponential function	<code>exp(2)</code>	7.3891
Natural logarithm	<code>log(exp(2))</code>	2
Base-10 logarithm	<code>log10(1000)</code>	3
Base-2 logarithm	<code>log2(16)</code>	4
Sine of $\pi/6$	<code>sin(pi/6)</code>	0.5
Cosine of $\pi/6$	<code>cos(pi/6)</code>	0.8660
Tangent of $\pi/6$	<code>tan(pi/6)</code>	0.5774

Answer

Input	round(x)	fix(x)	floor(x)	ceil(x)
-0.6	-1	0	-1	0
-0.3	0	0	-1	0
0.3	0	0	0	1
0.6	1	0	0	1

The Answer

```
x = -7.8;
y = 64;
z = 25;
theta = pi/6;
values = [-0.6 -0.3 0.3 0.6];

sqrt_z = sqrt(z)
root_value = nthroot(-27,3)

abs_x = abs(x)
sign_x = sign(x)

fact_5 = factorial(5)

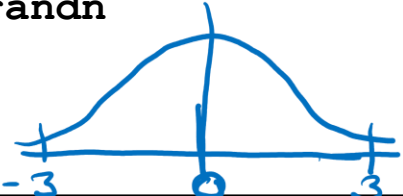
rem_25_4 = rem(25,4)
rem_9_5 = rem(9,5)

exp_2 = exp(2)
log_exp2 = log(exp(2))
log10_1000 = log10(1000)
log2_16 = log2(16)

sin_theta = sin(theta)
cos_theta = cos(theta)
tan_theta = tan(theta)

round_values = round(values)
fix_values = fix(values)
floor_values = floor(values)
ceil_values = ceil(values)
```

Random Numbers

Function	Description	Example	
		Script	Command Window
rand 	Returns a random value drawn from the uniform distribution on the interval (0,1).	<code>rand</code> <code>rand</code> <code>rand</code>	0.176 0.432 0.001
randn 	Returns a random value drawn from the standard normal distribution (mean of 0, standard deviation of 1).	<code>randn</code> <code>randn</code> <code>randn</code>	1.785 -17.6 0.15
randi([low,high]) 	Returns a random integer drawn from the uniform distribution on the interval [low,high].	<code>randi([20,77])</code> <code>randi([-50,0])</code> <code>randi([1,300])</code>	77 -25 253
rng(seed)	Specifies a seed for the random number generator.	<code>rng('default')</code> <code>rng('shuffle')</code> <code>rng(10694)</code>	

rng(#)
'Choose page
in random # not hard'

- [illegible]

📖: You can specify a seed. Then, the same numbers are generated.

📖: The shuffle seed is based on the current clock time.

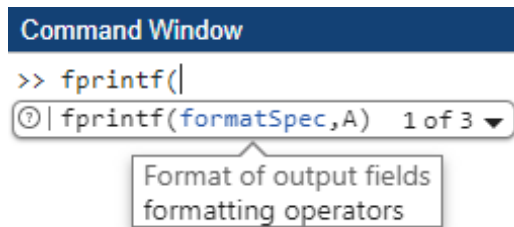
Seed:

Remembering All of These Functions

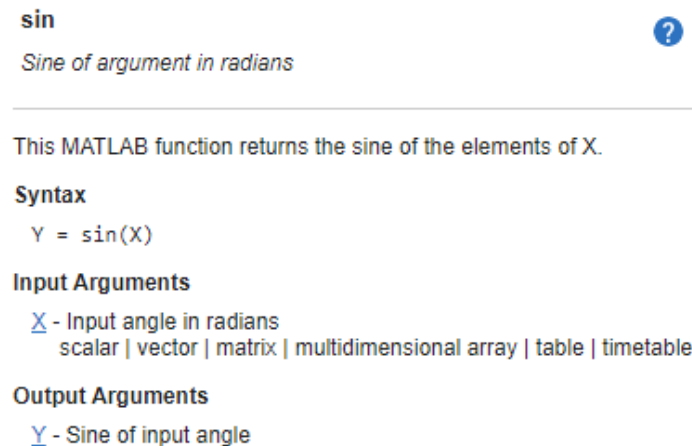
- There are way too many built-in functions in MatLab to remember all their names, operations, and syntax.
- Luckily, there MatLab has several reference tools to help support you as you code:

☺: The docs page can also be accessed in-program by right-clicking on the script and selecting “Help on Selection”.

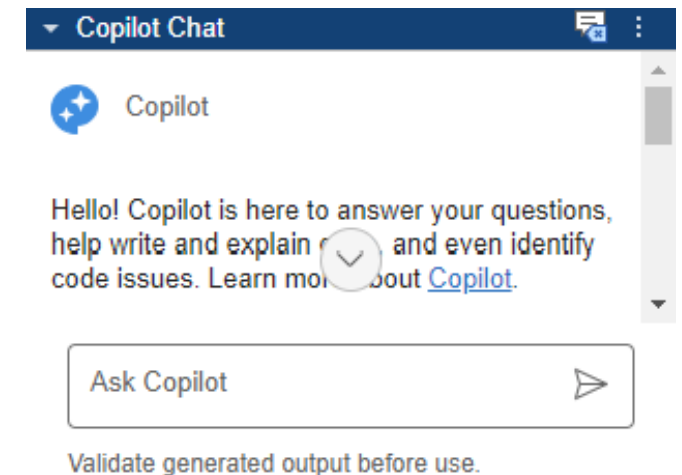
In-Line Argument Prompts



Online Documentation



Copilot



Example: Solving Complex Functions

Deliver the output of the following function for a randomly generated x and y terms on the interval (0,15).

$$z = \frac{(x^2 - 2y)^3}{100x} + \frac{|xy|}{5\sqrt{10x + y^{7/3}}} + \frac{\sin(x) \cos(y)}{5}$$

```
1 fprintf('\n%5s%5s%10s\n', 'x', 'y', 'z');
2 fprintf('%5s%5s%10s\n', '----', '----', '----');
3
4 x = rand*15;
5 y = rand*15;
6 z = (x^2-2*y)^3/(100*x) + ...
7     abs(x*y)/(5*sqrt(10*x+y^(7/3))) + ...
8     sin(x)*cos(y)/5;
9 fprintf('%5.1f%5.1f%10.4f\n', x,y,z)
```

x	y	z
----	----	----
3.7	10.0	-0.1068

Descriptive Inputs and Outputs

Input and output statements from the code should be sufficiently descriptive that the user doesn't need any extra information to use your program.

Helpful Inputs/Outputs

1	<code>year = input("What year is it?\n");</code>
2	<code>gradYear = year + 4;</code>
3	<code>fprintf("\nHello Class of %d!\n", gradYear);</code>

Unhelpful Inputs/Outputs

1	<code>year = input("Give the input.");</code>
2	<code>gradYear = year + 4;</code>
3	<code>fprintf("%d\n", gradYear);</code>



Comments

- All programming languages have the ability to insert lines of text that are not evaluated by the computer as commands in the code. These lines of text are called comments.
- In MatLab, comments look like this

1	% This is a comment.
2	var1 = 56.2; % This is also a comment.

- Comments are initiated with the % symbol. MatLab will indicate the comment is active by automatically turning the text green.
- Once a comment has been activated, MatLab will ignore all text until the end of the line.
- Whole lines/sections can be quickly commented out by selecting some text in the line and using the hotkey 'ctrl+R'. Likewise,

☺: Whole lines/sections can be quickly commented out by selecting some text in the line and using the hotkey 'ctrl+r'. Likewise, lines/sections of code can be uncommented with the hotkey 'ctrl+t'. These commands will come in handy for debugging.

When To Use Comments

Laying Out Preliminary Information

```
1 % Author: Your Name
2 % Purpose: Short description of what the
3 % code does
4 % Date Created: Date
5 % Acknowledgements: Any outside sources
6 % used to complete assignment, including
7 % website, books, and people (other than
8 % the teaching team)
```

Clarifying Variables

```
1 grav = 9.81; % gravitational acc. [m/s^2]
```

Specifying Code Structure

```
1 % ----- Gather Inputs -----
2 year = input("What year is it?\n");
3
4 % --- Calculate Graduation Date ---
5 gradYear = year + 4;
6
7 % ----- Greet the Class -----
8 fprintf("\nHello Class of %d!\n", ...
9 gradYear);
```

shortcut for mac on comments

Expectations for this Class

A Good Example

```
1 % Author: Alana Lund
2 % Purpose: To determine the roots and vertex of a parabola.
3 % Date Created: 11 May 2023
4
5 % Provide Context to the User
6 ...
7
8 % Take quadratic polynomial coefficients as inputs
9 ...
10
11 % Find Roots of the Polynomial
12 ...
13
14 % Find Vertex of the Polynomial
15 ...
16
17 % Print Outputs
18 ...
```

At a minimum, another developer (such as our TA) should be able to read the comments only the comments in your code and know exactly what logical steps you took to solve the problem.

: Laying out your comments before you write your code can help you break down the logical steps you need to take, and make writing the actual computations significantly easier.

Expectations for this Class

A Poor Example

```
1 % Author: Alana Lund
2 % Purpose: To calculate values from an expression.
3 % Date Created: 11 May 2023
4
5 % fprintf statement to start
6 ...
7
8 % Get a,b,c
9 ...
10
11 % Calculate  $-b \pm \sqrt{b^2 - 4ac} / (2a)$ 
12 ...
13
14 % Calculate  $-b / (2a)$  and  $a^2 + bx + c$ 
15 ...
16
17 % fprintf statement to end
18 ...
```

Your comments should not reflect the exact calculations of your code mathematically, nor should they be overly generalized.

Instead try to provide enough information that anyone reading your code can understand it without having to do the math.

MatLab as a Programming Tool

Instead of Just a Fancy Calculator

Environmental Engineering

Question:

Suppose wastewater flows in a pipe with a flow rate of **0.25 m³/s**. The pollutant concentration is **35 mg/L**. Compute the daily pollutant load in **kg/day**.

Geological Engineering

Question:

Suppose a rock layer has a density of **2100 kg/m³**. Estimate the vertical stress at a depth of **25 m**. Use gravitational acceleration **$g = 9.81 \text{ m/s}^2$** and report the answer in **kPa**.

[MATLAB as a Calculator](#)

[MATLAB as a Programming Tool](#)

MatLab as a Programming Tool

Instead of Just a Fancy Calculator

Environmental Engineering

Question:

Suppose wastewater flows in a pipe with a flow rate of **0.25 m³/s**. The pollutant concentration is **35 mg/L**. Compute the daily pollutant load in **kg/day**.

MATLAB as a Calculator

</> MATLAB

```
0.25 * 35 * 0.001 * 86400
```

Output:

</> MATLAB

```
ans =
```

```
756
```

MATLAB as a Programming Tool

</> MATLAB

```
% MATLAB as a programming tool
% Set constants
Q = 0.25;           % flow rate in m^3/s
C = 35;             % concentration in mg/L
mgL2kgm3 = 0.001;  % convert mg/L to kg/m^3
sec2day = 86400;    % seconds in one day

% Compute pollutant load
load = Q * C * mgL2kgm3 * sec2day;

% Report result to user
fprintf('The daily pollutant load is %.2f kg/day.\n', load);
```

Expected result table:

Name	Value
Q	0.25
C	35
mgL2kgm3	0.001
sec2day	86400
load	756

MatLab as a Programming Tool

Instead of Just a Fancy Calculator

Geological Engineering

Question:

Suppose a rock layer has a density of **2100 kg/m³**. Estimate the vertical stress at a depth of **25 m**. Use gravitational acceleration **$g = 9.81 \text{ m/s}^2$** and report the answer in **kPa**.

MATLAB as a Programming Tool

MATLAB as a Calculator

```
</> MATLAB
```

```
2100 * 9.81 * 25 / 1000
```

Output:

```
</> MATLAB
```

```
ans =
```

```
515.0250
```

```
</> MATLAB
```

```
% MATLAB as a programming tool
```

```
% Set constants
```

```
density = 2100; % rock density in kg/m^3
```

```
g = 9.81; % gravitational acceleration in m/s^2
```

```
depth = 25; % depth in m
```

```
Pa2kPa = 1000; % convert Pa to kPa
```

```
% Compute vertical stress
```

```
stress = density * g * depth / Pa2kPa;
```

```
% Report result to user
```

```
fprintf('The vertical stress at %.1f m depth is %.2f kPa.\n', depth, stress);
```

Expected result table:

Name	Value
density	2100
g	9.81
depth	25
Pa2kPa	1000
stress	515.025